A NOVEL WEB-BASED METHODOLOGY FOR OPTIMIZING KNOX:- AI POWERED CHARACTER CREATION AND CONVERSATIONAL PLATFORM USING GEMINI API

A PROJECT REPORT SUBMITTED

IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR

THE AWARD OF THE DEGREE

OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

BY

Shubham Kumar	**22050440093**
Abhinav Mishra	**22050440003**
Rohit Munda	**22050440084**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING CAMBRIDGE

INSTITUTE OF TECHNOLOGY

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **A novel web-based methodology for optimizing KNOX AI – powered character creation and conversational platform using Gemini API** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

Prof. Sonal Tudu
Dept. of Computer Science and Engineering
Cambridge Institute of Technology Tatisilwai,
Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science and Engineering
Cambridge Institute of Technology Tatisilwai,
Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled **Knox: A Novel Web-Based Methodology for AI-Powered Character Creation and Intelligent Conversational Interaction Using Gemini API** is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner) **(External Examiner)**

HEAD OF DEPARTMENT

Dept. of Computer Science and Engineering

Cambridge Institute of Technology

Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

we take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. we would like to express my sincere thanks to **Prof. Deepak Kumar Verma** (HOD, Department of Computer Science & Engineering) and to my guide **Prof. Sonal Tudu**, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal.

we would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. we have been fortunate to have a wonderful support structure among fellow students. our sincere thanks to everyone who has provided us with kind words, new ideas, useful criticism, or their valuable time. we are truly indebted. Last but not least, we would like to dedicate this work to our family for their love and understanding.

SHUBHAM KUMAR
22050440093
ABHINAV MISHRA
22050440003
ROHIT MUNDA
22050440084

ABSTRACT

The rapid integration of Artificial Intelligence into modern web applications has opened new avenues for building personalized, context-aware user experiences. However, most existing conversational systems remain generic in nature, offering little room for customization or meaningful user engagement. This project introduces Knox, an AI-powered web platform designed to let users create, customize, and converse with intelligent virtual characters tailored to their individual preferences.

Knox is built on a full-stack architecture comprising React.js, Express.js, Node.js, and MongoDB, with Tailwind CSS handling the user interface. Authentication is secured through JSON Web Tokens (JWT), and Role-Based Access Control (RBAC) is implemented to manage user and administrative privileges. The platform leverages the Google Gemini API to generate intelligent, context-sensitive responses, while MongoDB stores conversation history that is fed back into the model as context — enabling more coherent and consistent dialogue over time.

Core features of the platform include user registration and login, AI character creation with customizable personalities, a dedicated chat interface, secure session management, and persistent chat history. The system follows a modular design philosophy, prioritizing scalability, maintainability, and performance.

Evaluation of the platform confirms that Knox successfully delivers personalized AI-driven interactions while upholding security standards and contextual accuracy. This project demonstrates a practical approach to integrating large language models into production-ready web applications, and lays the groundwork for future improvements including long-term memory, voice interaction, multilingual support, and richer character customization.

Keywords: Artificial Intelligence, Gemini API, Conversational AI, Character Creation, MongoDB, React.js, Express.js, Context-Aware Chatbot.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

API	Application Programming Interface
DFD	Data Flow Diagram
ER	Entity Relationship
IDE	Integrated Development Environment
SQL	Structured Query Language
UAT	User Acceptance Testing
UML	Unified Modelling Language
UI	User Interface

CHAPTER 1 – INTRODUCTION

1.1 Background and Motivation

Artificial Intelligence has changed the way people interact with technology, making natural language communication with digital systems more accessible than ever. Yet most existing chatbot platforms remain generic — they respond, but they don't feel personal. Users have little to no control over how an AI behaves, what personality it carries, or how it remembers past conversations.

This gap is what Knox was built to fill. Knox is an AI-powered web platform that lets users create, customize, and chat with their own intelligent virtual characters — whether for companionship, learning, role-playing, or productivity. It integrates the Google Gemini API for intelligent responses, stores chat history in MongoDB for conversational continuity, and uses JWT-based Role-Based Authentication to keep the platform secure and well-managed. The tech stack includes React.js, Express.js, Node.js, MongoDB, and Tailwind CSS.

1.2 Problem Statement

Most chatbot systems offer little more than scripted or one-size-fits-all responses. Users cannot define personalities, maintain context across conversations, or manage access in a meaningful way. Knox addresses this directly by allowing users to build custom AI characters with unique personalities, backed by context-aware conversations, secure authentication, and proper user management.

1.3 Objectives of the Project

The key objectives of Knox are:

- Build an AI-powered platform for character creation and conversation
- Allow users to create and customize virtual AI characters
- Deliver context-aware responses using the Gemini API
- Store chat history to maintain conversational continuity
- Implement secure JWT-based authentication and authorization
- Apply Role-Based Access Control (RBAC) for user management

1.4 Scope of the Project

User Management

The platform supports a complete user lifecycle — from registration to login and session management. During registration, users provide basic credentials which are securely stored in MongoDB with password hashing applied before saving. Login is handled through a validated authentication flow that issues a JSON Web Token (JWT) upon successful verification. This token is then used to authenticate all subsequent requests made by the user, ensuring that only verified users can access protected platform features.

Role-Based Access Control (RBAC) is implemented to distinguish between two primary user roles — standard users and administrators. Standard users have access to character creation, chat interactions, and personal account management. Administrators hold elevated privileges that allow them to oversee and manage the broader platform, including user accounts and character data.

AI Character Creation and Management

One of the core features of Knox is the ability for users to create their own intelligent virtual characters. During character creation, users can define the character's name, personality description, and behavioral traits. These details are stored in MongoDB and are later used to shape how the AI responds during conversations — essentially acting as a system-level prompt that guides the Gemini model's behavior.

Users have full control over their characters and can edit character details or delete characters they no longer need. This gives each user a personalized roster of AI agents they can interact with at any time, each behaving differently based on how they were configured.

Context-Aware Conversations

Knox integrates the Google Gemini API as its core AI engine. When a user sends a message to a character, the platform does not simply forward that single message to the model. Instead, it retrieves the most recent conversation exchanges from MongoDB and bundles them together with the character's personality description and the new user message before sending the request to Gemini. This approach ensures that the AI has enough context to generate responses that are relevant, coherent, and consistent with the character's defined personality.

This context management mechanism significantly improves the quality of conversations compared to stateless chatbot systems that treat every message in isolation.

Chat History Storage

Every conversation between a user and an AI character is stored persistently in MongoDB. Each message is recorded along with metadata such as the sender, timestamp, and the character involved. This serves two purposes — it allows users to scroll back and review past conversations through the chat interface, and it provides the data needed for the context injection mechanism described above.

Chat history is organized per user and per character, meaning that conversations with different characters are kept separate and do not interfere with one another.

Secure Authentication and Session Handling

Security is a key consideration throughout the platform. JWT-based authentication ensures that user sessions are stateless and tamper-proof. Upon login, a signed token is issued to the client and stored securely. This token must accompany every request to a protected API route, where it is verified by the server before any data is returned or action is performed.

Sensitive data such as passwords are hashed using industry-standard hashing before being stored in the database, ensuring that raw credentials are never persisted. Protected routes on both the frontend and backend enforce authentication checks, preventing unauthorized access to user data or platform functionality

Admin Controls

Administrators have access to a dedicated layer of platform management. Through admin-level access, they can view and manage registered users, monitor character data across the platform, and take action where necessary — such as removing inappropriate content or deactivating user accounts. This role separation ensures that the platform remains organized and that general users cannot interfere with data that does not belong to them.

Future Scope and Expansion Possibilities

Long-Term Memory Currently, the platform supplies only recent chat history as context to the AI model. A future enhancement would involve implementing a long-term memory system that allows AI characters to retain important information about a user across multiple sessions — such as their name, preferences, past topics discussed, and behavioural patterns. This would make interactions feel significantly more personal and continuous over time.

Voice Interaction Integrating speech-to-text and text-to-speech capabilities would allow users to converse with their AI characters through voice rather than typing. This would make the platform more accessible and create a more immersive conversational experience, particularly for users who prefer hands-free interaction.

Multilingual Support Adding support for multiple languages would open the platform to a much wider global audience. Users would be able to create characters that converse in their preferred language, and the interface itself could be localized to improve usability for non-English speakers.

Avatar and Image Generation Future versions could allow users to generate or upload visual avatars for their AI characters. Integrating image generation APIs would enable dynamic, AI-generated character visuals that match the personality and traits defined by the user, adding a strong visual identity to each character.

1.5 Organization of the Report

Chapter 1 – Introduction presents the background and motivation behind the Knox project, outlines the problem being addressed, defines the objectives, and establishes the scope of the current system. It provides the reader with the necessary context to understand why the platform was built and what it aims to achieve.

Chapter 2 – Literature Review examines existing AI-powered conversational platforms and character-based chatbot systems. It analyses the strengths and limitations of current solutions, reviews the underlying technologies such as large language models and conversational AI frameworks, and establishes the justification for the approach taken in developing Knox.

Chapter 3 – System Analysis and Design covers the functional and non-functional requirements of the platform, system architecture, database design, and the overall flow of the application. This chapter includes use case diagrams, data flow diagrams, and entity-relationship diagrams that illustrate how the different components of Knox interact with one another.

Chapter 4 – Implementation details the actual development of the platform, covering the technologies and tools used, the structure of the frontend and backend, integration of the Google Gemini API, JWT authentication implementation, Role-Based Access Control, and the chat context management mechanism. Key code logic and design decisions are explained in this chapter.

Chapter 5 – Testing and Evaluation presents the testing strategy adopted for the project, including functional testing, authentication and security testing, and API response validation. Test cases, results, and observations are documented to demonstrate that the system meets its intended objectives and performs reliably under expected conditions.

Chapter 6 – Conclusion and Future Enhancements summarises the outcomes of the project, reflects on what was achieved, and discusses the limitations of the current version. It also outlines the planned future enhancements that would extend the capabilities of Knox beyond its current scope.

CHAPTER 2 – LITERATURE REVIEW

2.1 Related Works

The rise of Artificial Intelligence and Natural Language Processing (NLP) has fundamentally changed how people interact with software. What once required carefully scripted rule-based logic can now be handled by Large Language Models (LLMs) that understand context, generate human-like responses, and hold meaningful conversations. Technologies like Google's Gemini and OpenAI's GPT series sit at the centre of this shift, and their growing accessibility through APIs has made it practical for developers to build intelligent conversational applications without training models from scratch..

Character.AI is one of the most widely used character-based conversational platforms available today. It allows users to interact with AI personas that carry distinct personalities and conversation styles. The experience it delivers is genuinely engaging, particularly for roleplay and entertainment. However, from a development standpoint, the platform offers very little control. Users interact with pre-existing or community-created characters but have limited ability to define system-level behavior, manage access, or integrate the platform into a broader application architecture.

Replika AI takes a different approach, focusing on building a long-term AI companion that learns from its conversations with a specific user over time. It does emotional engagement well and has developed a loyal user base. That said, its purpose is narrow — it is built around companionship, and it does not support the kind of multi-character management or role-based user control that a platform like Knox requires.

Traditional rule-based chatbots still exist across many websites and customer service tools. They are easy to implement and predictable in behavior, but they struggle with anything outside their scripted paths. A question phrased slightly differently than expected can completely break the conversation, which makes them feel rigid and frustrating to interact with.

LLM-based systems powered by models like Gemini or GPT represent the current standard for intelligent conversation. They handle context well, generate natural responses, and adapt to a wide range of topics. The main consideration is that they depend on external API services, which introduces factors like rate limits, latency, and cost into the application design.

2.2 Comparative Analysis

Traditional chatbots are still useful in narrow, well-defined scenarios — answering FAQs, guiding users through a fixed process, or handling simple support queries. But the moment a conversation steps outside those boundaries, they break down. They have no real understanding of language, no memory of what was said earlier, and no ability to adapt. For a platform like Knox, where the goal is meaningful and personalized interaction, a rule-based approach simply would not work.

LLM-powered platforms solve the intelligence problem well. Character.AI and Replika both prove that users respond positively to AI that feels personal and contextually aware. However, both platforms are closed ecosystems. A user can interact with characters, but they cannot build their own characters with custom system behavior, manage multiple users with different access levels, or control how conversation history is handled under the hood. These are not minor gaps — they are fundamental limitations for anyone wanting to use such a platform for education, development, or structured application deployment.

system rather than added as an afterthought. Standard users manage their own characters and conversations, while administrators have oversight of the broader platform. This separation of access is something neither Character.AI nor Replika offers in a developer-accessible way.

The use of React.js, Express.js, Node.js, MongoDB, and Tailwind CSS gives Knox a modern, maintainable foundation that is well suited for future expansion. The architecture is modular enough to support new features without requiring significant restructuring, which matters when planning enhancements like long-term memory, voice interaction, or real-time messaging down the line.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM REQUIREMENTS

The successful implementation of KNOX requires a combination of hardware resources, software tools, and supporting technologies. Since the platform is a web-based AI conversational application, the system must be capable of handling user authentication, character management, database operations, and real-time communication with the Gemini API. The requirements listed below were identified during the development phase to ensure smooth performance, security, and scalability of the application.

3.1.1 Hardware Requirements

The hardware requirements for developing and running the application are relatively moderate because most of the data storage and AI processing tasks are handled through cloud services such as MongoDB Atlas and Gemini API.

Processor	**RAM**	**Storage**	**Internet Connection**	**Display**	**Input Devices**
Intel core i3(8^{th}generation) or above	Minimum 4 GB (8 GB recommended)	20 GB of available disk space	Stable broadband connection	1366 × 768 resolution or higher	Standard Keyboard and Mouse

3.1Table Hardware Requirements

During development, a system with at least 8 GB RAM was preferred to efficiently run the frontend server, backend server, and database-related operations simultaneously. A stable internet connection is essential because the application continuously interacts with cloud-based services.

-

3.1.2 Software Requirements

The KNOX platform was developed using modern web technologies and follows the MERN stack architecture. Various software tools and frameworks were used to build different components of the system.

Operating System	**Frontend Framework**	**Backend Framework**	**Backend Runtime**	**Backend Framework**	**Database**
Windows 10/11, Linux, or macOS	React.js	React.js	Node.js	Express.js	MongoDB Atlas

Table 3.1.2 – Software Requirements

Styling Framework	**Authentication**	**AI Service**	**Version Control**	**Code Editor**	**Web Browser**
Tailwind CSS	JSON Web Token (JWT)	Google Gemini API	Git and GitHub	Visual Studio Code	Google Chrome, Microsoft Edge, Firefox

Table 3.1.2 – Software Requirements

The frontend was developed using React.js because of its component-based architecture and efficient rendering capabilities. Tailwind CSS was chosen to create responsive and visually appealing user interfaces. Express.js and Node.js were used to build RESTful APIs and handle server-side operations. MongoDB Atlas provides cloud-based storage for user accounts, character information, and chat data.

3.1.3 Functional Requirements

Functional requirements describe the major features and services that the system must provide to users.

User Registration and Authentication

The system must allow new users to create accounts using their credentials. Registered users should be able to log in securely and access their personalized dashboard. Authentication is implemented using JWT tokens to maintain secure user sessions.

Character Creation and Management

Users must be able to create custom AI characters by providing details such as character name, personality, description, and other attributes. Users should also have the ability to manage and update their created characters.

AI-Powered Conversations

The application must allow users to interact with AI-generated characters through a chat interface. Messages sent by users are processed using the Gemini API, which generates relevant responses based on the provided context.

Conversation Context Management

To improve response quality and maintain conversation continuity, the system must retrieve and send the previous ten messages along with the current message to the AI model. This enables the AI to understand the flow of the conversation and generate more contextually accurate replies.

Role-Based Access Control (RBAC)

The system must implement Role-Based Access Control to restrict access to specific functionalities based on user roles. RBAC helps improve security and ensures that only authorized users can perform certain actions.

Chat History Storage

The system should store conversation data in the database so that users can continue previous interactions without losing important information.

Responsive User Interface

The application should provide a responsive design that works efficiently across desktops, laptops, tablets, and mobile devices.

3.1.4 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance

The application should provide quick response times for authentication requests, character management operations, and AI-generated conversations. Efficient API communication and optimized database queries are necessary to achieve satisfactory performance.

Scalability

The architecture should support future growth in the number of users, characters, and conversations without significant degradation in performance.

Reliability

The system should operate consistently and handle unexpected failures gracefully. Cloud-based services such as MongoDB Atlas help improve system reliability and data availability.

Security

User credentials and sensitive information must be protected from unauthorized access. JWT-based authentication, password encryption, secure API communication, and RBAC mechanisms contribute to overall system security.

Usability

The user interface should be simple, intuitive, and easy to navigate. Users should be able to register, create characters, and start conversations without requiring technical knowledge.

Maintainability

The application should be developed using a modular structure so that future enhancements, bug fixes, and feature additions can be implemented easily.

Availability

Since KNOX depends on cloud services, the application should remain accessible whenever internet connectivity is available. MongoDB Atlas and Gemini API provide high availability for database and AI operations.

3.1.5 Development Environment

The development of KNOX was carried out using Visual Studio Code as the primary code editor. Git and GitHub were used for source code management and collaboration. MongoDB Atlas served as the cloud database, while Gemini API was integrated for generating AI responses. Testing was performed using modern web browsers to ensure compatibility and responsiveness across different platforms.

3.2 SYSTEM ARCHITECTURE

The architecture of KNOX is designed using the MERN (MongoDB, Express.js, React.js, and Node.js) stack and follows a client-server model. The system integrates modern web technologies with Generative Artificial Intelligence to provide users with an interactive character-based conversational experience. The architecture has been designed to ensure scalability, security, maintainability, and efficient communication between different system components.

KNOX consists of four major layers:

1. Presentation Layer (Frontend)
2. Application Layer (Backend)
3. Data Layer (Database)
4. Artificial Intelligence Layer (Gemini API)

These layers work together to process user requests, manage application data, authenticate users, and generate AI-powered responses.

3.2.1 Architecture Overview

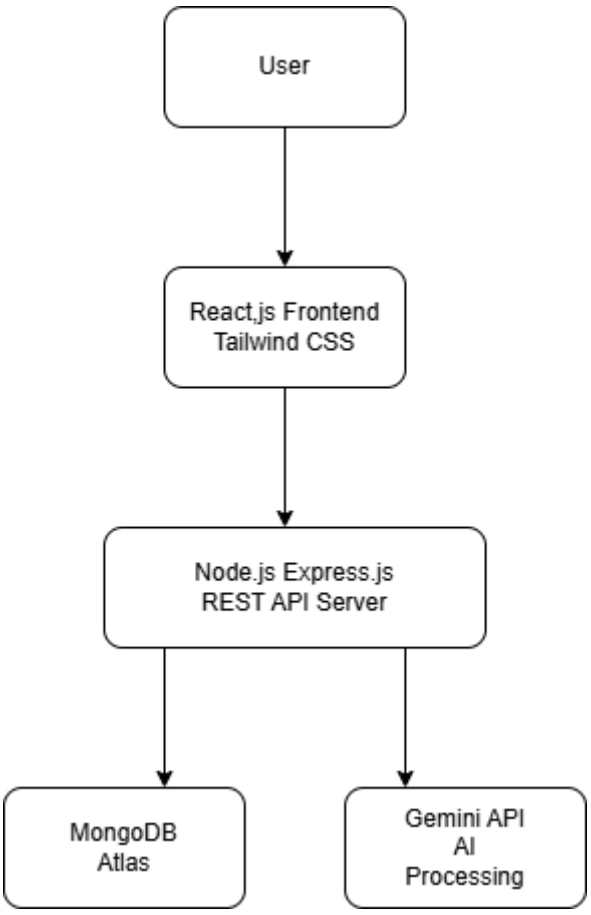
The user accesses the application through a web browser. The frontend, developed using React.js and Tailwind CSS, provides interfaces for user registration, login, character creation, and chat interactions. User actions are sent to the backend through RESTful APIs.

The backend is developed using Node.js and Express.js. It handles business logic, authentication, authorization, character management, chat processing, and communication with external services. The backend also verifies JWT tokens and

enforces Role-Based Access Control (RBAC) to ensure secure access to application resources.

MongoDB Atlas serves as the cloud database where user information, character details, and conversation histories are stored. This enables persistent data storage and retrieval whenever users access the platform.

The Gemini API acts as the intelligence layer of the system. Whenever a user sends a message, the backend retrieves the last ten messages from the conversation and includes them as context before sending the request to Gemini. The AI model processes this information and generates a context-aware response, which is then returned to the user.



3.2.2 System Architecture Diagram

3.3 Data Flow Diagrams

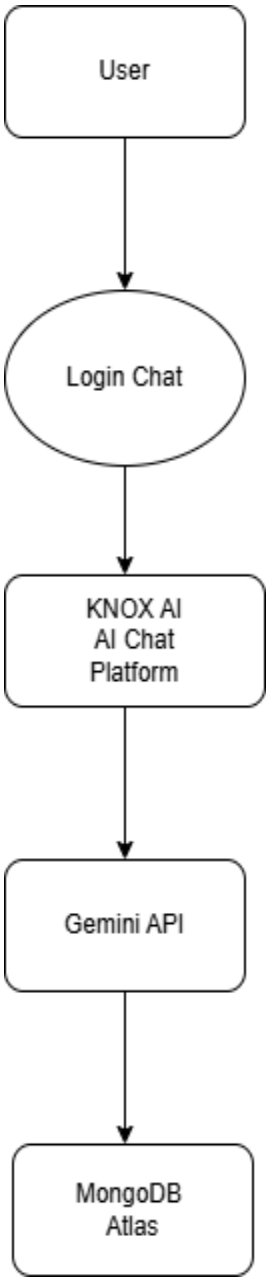


Figure 3.2 – Data Flow Diagram (Level 0)

Description of above table

The user interacts with the KNOX platform for authentication, character creation, and chatting. The platform communicates with MongoDB Atlas for data storage and Gemini API for generating AI responses.

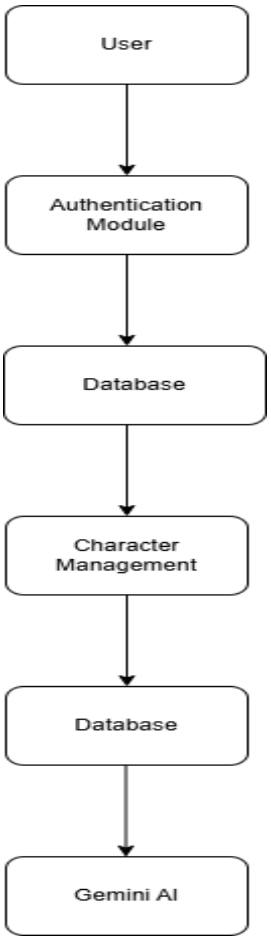


Figure 3.3 – Data Flow Diagram (Level 1)

Description

The Level 1 DFD divides the system into three main modules:

- Authentication Module
- Character Management Module
- Chat Processing Module

Each module communicates with the database, while the chat module additionally interacts with Gemini AI

3.4 ER Diagram / UML Diagrams

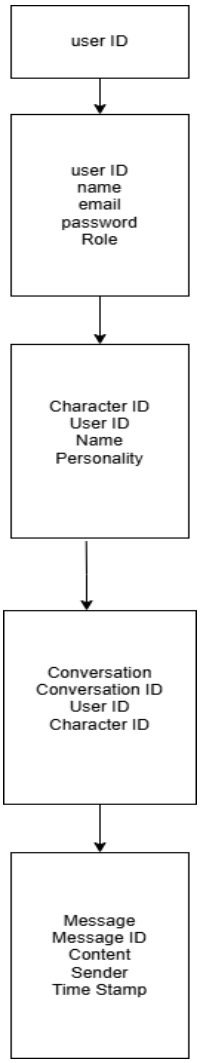


Figure 3.4-Entity Relationship Diagram

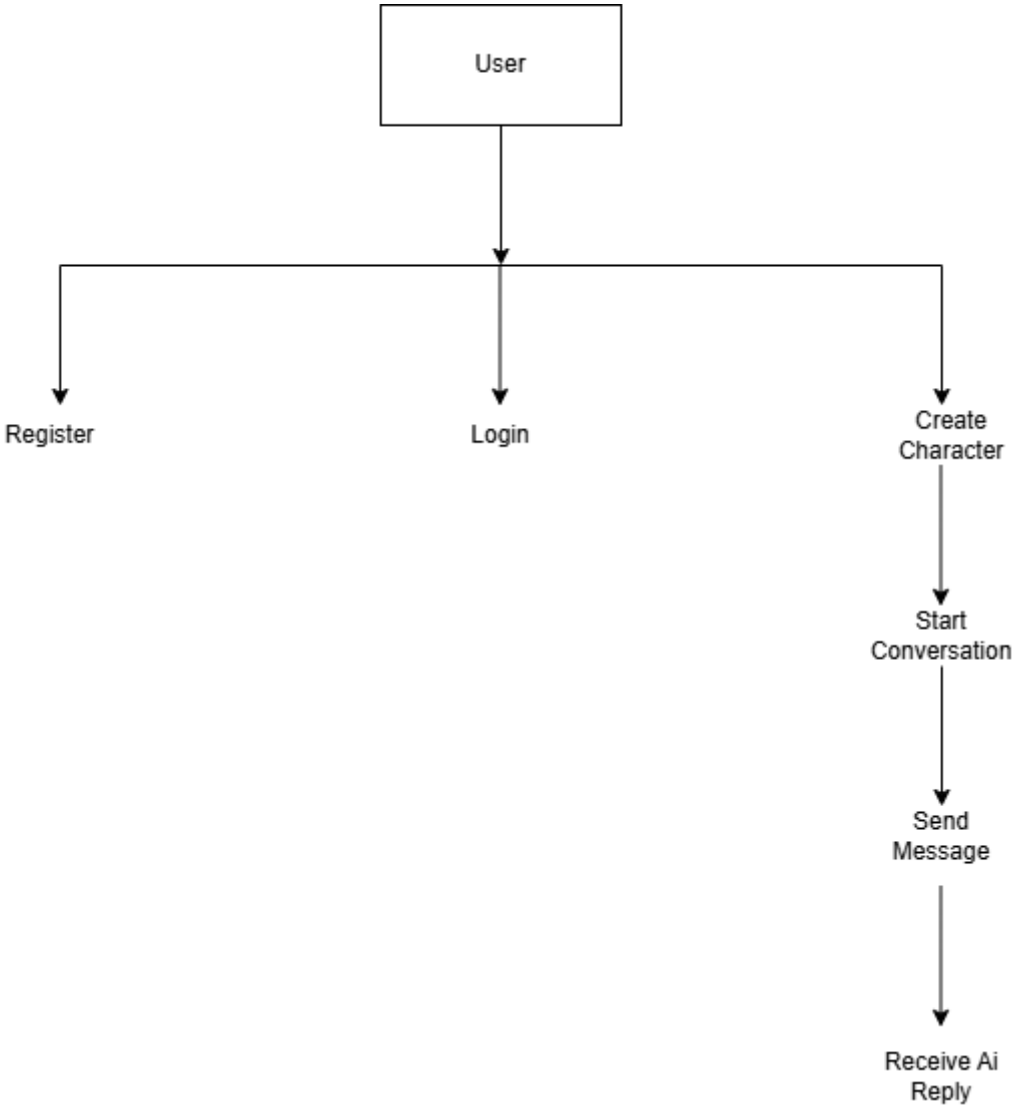


Figure 3.5-Use case Diagram

CHAPTER 4 – IMPLEMENTATION

4.1 Technologies and Tools Used

The KNOX platform was developed using modern web development technologies and cloud services. The selection of each technology was based on factors such as performance, scalability, ease of development, community support, and compatibility with Artificial Intelligence integration.

4.1.1 React.js

React.js was used for frontend development. It provides a component-based architecture that simplifies the creation of reusable user interface elements. React's virtual DOM improves rendering performance and enables smooth user interactions.

Reasons for Selection:

- Fast rendering through Virtual DOM.
- Reusable components reduce development effort.
- Strong community support.
- Easy integration with REST APIs.

4.1.2 Tailwind CSS

Tailwind CSS was used for styling the user interface. It allows rapid development of responsive layouts using utility-based CSS classes.

Reasons for Selection:

- Faster UI development.
- Responsive design support.
- Reduced CSS file complexity.
- Modern and clean user interface design.

4.1.3 Node.js

Node.js was used as the backend runtime environment. It enables server-side JavaScript execution and handles asynchronous operations efficiently.

Reasons for Selection:

• Non-blocking architecture.
• High performance.
• Suitable for real-time applications.
• Easy integration with JavaScript frontend technologies.

4.1.4 Express.js

Express.js was used to develop RESTful APIs and manage server-side routing.

Reasons for Selection:

• Lightweight framework.
• Simplified route management.
• Middleware support.
• Easy integration with MongoDB and authentication systems.

4.1.5 MongoDB Atlas

MongoDB Atlas was selected as the cloud database solution for storing user accounts, character information, and conversation history.

Reasons for Selection:

• Flexible NoSQL database structure.
• Cloud-based accessibility.
• Automatic backups and monitoring.
• Scalable architecture.

4.1.6 Gemini API

The Gemini API serves as the core Artificial Intelligence engine of the application. It generates intelligent and context-aware responses for user conversations.

Reasons for Selection:

• Advanced natural language processing capabilities.
• Fast response generation.

4.1.7 JSON Web Token (JWT)

JWT is used for authentication and authorization purposes.

Reasons for Selection:

• Secure token-based authentication.
• Stateless session management.
• Lightweight implementation.
• Improved security for API communication.

4.1.8 Development Tools

Visual Studio Code:
Used as the primary code editor.

Git and GitHub:
Used for version control and project management.

Postman:
Used for testing backend APIs.

Google Chrome:
Used for frontend testing and debugging.

4.2 Module Description

The KNOX platform is divided into multiple modules, each responsible for a specific functionality within the system.

4.2.1 User Authentication Module

Purpose:

This module manages user registration, login, and session validation.

Input:

• User Name
• Email Address
• Password

Process:

• Validates user information.
• Encrypts passwords before storage.
• Generates JWT tokens after successful login.

Output:

• User account creation.

• Secure login session.

Dependencies:

• MongoDB Atlas

• JWT Authentication

4.2.2 Character Management Module

Purpose:

Allows users to create and manage AI characters.

Input:

• Character Name

• Character Description

• Personality Traits

Process:

• Stores character information.

• Associates characters with the user account.

Output:

• Personalized AI character profiles.

Dependencies:

• MongoDB Atlas

Figure 4.2 – Character Creation Screen

4.2.3 Chat Processing Module

Purpose:

Handles message exchange between users and AI characters.

Input:

• User Message

Process:

• Retrieves previous conversation messages.
• Collects the last ten messages as context.
• Sends context and current message to Gemini API.

Output:

• AI-generated response.

Dependencies:

• Gemini API
• Conversation Database

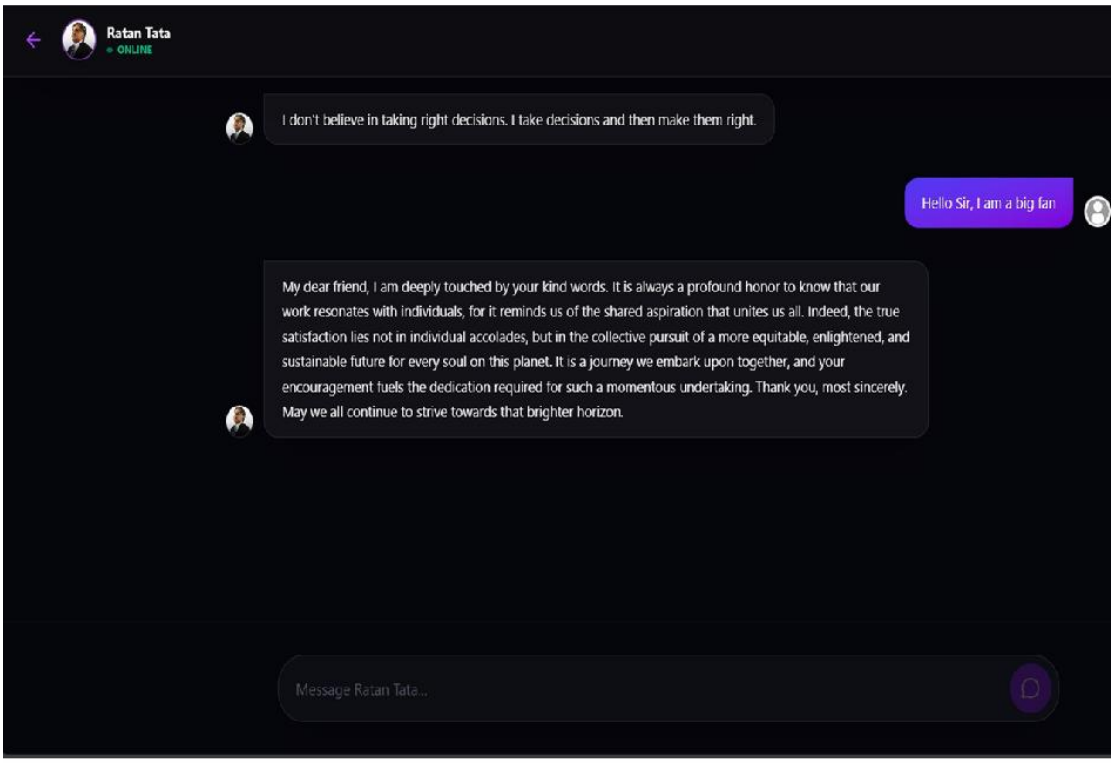


Figure 4.2 – Chat Interface

4.2.4 Context Management Module

Purpose:

Maintains conversational continuity by providing previous messages to the AI model.

Input:

• Current conversation history.

Process:

• Retrieves the latest ten messages.
• Formats them for API submission.

Output:

• Context-enriched AI request.

4.2.5 Role-Based Access Control (RBAC) Module

Purpose:

Controls access to system resources based on user roles.

Input:

• User Role
• API Request

Process:

• Verifies permissions.
• Grants or denies access.

Output:

• Authorized resource access.

Benefits:

• Enhanced security.
• Controlled system operations.

4.3 Algorithm / Methodology

The conversational workflow implemented in KNOX follows a context-aware AI response generation methodology.

Algorithm: Context-Aware Response Generation

Step 1:
Receive user message.

Step 2:
Validate user authentication token.

Step 3:
Retrieve conversation history.

Step 4:

Select the most recent ten messages.

Step 5:

Combine retrieved messages with current user input.

Step 6:

Send the complete context to Gemini API.

Step 7:

Receive generated response.

Step 8:

Store both user message and AI response in the database.

Step 9:

Return response to frontend.

Pseudo Code:

BEGIN

Receive User_Message

IF User_Authenticated THEN

Fetch Last_10_Messages

Context = Last_10_Messages + User_Message

AI_Response = Gemini_API(Context)

Store User_Message

Store AI_Response

Return AI_Response

ELSE

Return Authentication_Error

END IF

END

Time Complexity Analysis

Database Retrieval:
O(n), where n = number of messages retrieved (limited to 10).

Context Preparation:
O(n)

API Communication:
Depends on Gemini API processing time.

Overall Complexity:
O(n)

Since n is restricted to ten messages, the practical execution time remains nearly constant.

4.4 CODE IMPLEMENTATION

The complete source code is provided in Appendix A. Only key implementation snippets are included here.

User Authentication Route

```
router.post("/login", async (req, res) => {


 const { email, password } = req.body;


 const user = await User.findOne({ email });
```

```
if (!user) {
    return res.status(404).json({
        message: "User not found"
    });
}

const token = jwt.sign(
    { id: user._id },
    process.env.JWT_SECRET
);

res.json({ token });

});
```

Explanation:

This route validates user credentials and generates a JWT token upon successful authentication.

Context Retrieval Logic

```
const messages = await Message.find({
 conversationId
})
```

```
.sort({ createdAt: -1 })

.limit(10);
```

Explanation:

The query retrieves the ten most recent messages from a conversation to maintain AI context.

Gemini API Request

```
const response = await model.generateContent(

   conversationContext

);
```

Explanation:

The prepared conversation context is sent to Gemini API, which generates an AI response based on the conversation history.

Character Creation Example

```
const character = await Character.create({

name,

  description,

  personality,

  createdBy: userId

});
```

CHAPTER 5 – TESTING AND RESULTS

5.1 Testing Strategy

Testing plays a crucial role in software development as it helps ensure that the application functions correctly, meets user requirements, and performs reliably under different conditions. For the KNOX platform, a structured testing approach was followed throughout the development process. Various levels of testing were performed to identify and resolve issues related to functionality, integration, security, and user experience.

The primary objective of testing was to verify that all modules of the system work as expected and that users can interact with the application without encountering errors or security vulnerabilities.

5.1.1 Testin g Approach

The testing process for KNOX was divided into the following phases:

1. Unit Testing
2. Integration Testing
3. System Testing
4. User Acceptance Testing (UAT)

Each testing phase focused on a specific aspect of the application and contributed to the overall quality of the system.

5.1.2 Unit Testing

Unit testing was performed to verify the functionality of individual components and modules of the application. Each function, API endpoint, and database operation was tested independently to ensure that it produced the expected output for a given input.

The following components were tested during unit testing:

• User registration functionality
• Login authentication process
• JWT token generation and validation
• Character creation functionality
• Database operations

• API response handling

• Gemini API request generation

The purpose of unit testing was to identify coding errors at an early stage of development and ensure that individual modules behaved correctly before integration.

Example

Input:

Valid user registration details

Expected Output:

New user account created successfully in the database.

Actual Output:

User account created successfully.

Status:

Pass

5.1.3 Integration Testing

Integration testing was conducted after the successful completion of unit testing. The objective was to verify communication between different modules and ensure seamless data flow across the application.

The following integrations were tested:

• Frontend and Backend communication

• Backend and MongoDB Atlas integration

• Backend and Gemini API integration

• Authentication middleware and protected routes

• Character management and database interaction

• Chat module and AI response generation

Particular attention was given to testing the interaction between the chat processing module and the Gemini API, as this represents the core functionality of the application.

Example

Input:

User sends a chat message.

Expected Output:

Message is sent to Gemini API and a valid AI response is returned.

Actual Output:

AI response generated successfully.

Status:

Passed

5.1.4 System Testing

System testing was performed on the fully integrated application to verify that all functional and non-functional requirements were satisfied.

The complete system was tested in a production-like environment to evaluate:

• Authentication and authorization mechanisms

• Character creation and management

• AI-powered conversations

• Database operations

• User interface responsiveness

• Error handling mechanisms

• Security features

• Context management using previous messages

During system testing, different user scenarios were executed to ensure that the platform behaved correctly under normal operating conditions.

Features Tested

1. User Registration
2. User Login
3. JWT Authentication
4. Character Creation
5. Character Selection
6. Chat Functionality
7. Conversation History Storage
8. Context Retrieval (Last 10 Messages)
9. Role-Based Access Control (RBAC)
10. Logout Functionality

All major features operated successfully without critical defects.

5.1.5 User Acceptance Testing (UAT)

User Acceptance Testing was performed to determine whether the application meets user expectations and project objectives.

A small group of users was asked to interact with the application and provide feedback regarding:

• Ease of navigation

• User interface design

• Character creation process

• Quality of AI-generated responses

• Overall user experience

Users were able to register accounts, create AI characters, and conduct conversations without significant difficulties. Feedback received during UAT was used to make minor improvements to the interface and user experience.

UAT Outcome

Most users found the platform intuitive and easy to use. The context-aware response mechanism significantly improved conversation quality, resulting in a positive user experience.

5.1.6 Testing Tools Used

Several testing tools were used during the development and testing phases of the project.

Postman

Postman was used to test backend REST APIs independently of the frontend application.

Purpose:

• API testing

• Request and response validation

• Authentication testing

• Error response verification

Google Chrome Developer Tools

Chrome Developer Tools were used for frontend debugging and performance monitoring.

Purpose:

• JavaScript debugging
• Network request monitoring
• Responsive design testing
• Performance analysis

MongoDB Atlas Monitoring

MongoDB Atlas monitoring tools were used to verify database connectivity and query execution.

Purpose:

• Database testing
• Query validation
• Data verification

Manual Testing

Manual testing was performed throughout development to verify user workflows and identify unexpected issues.

Purpose:

• Functional testing
• UI validation
• User experience evaluation

5.1.7 Security Testing

Since KNOX handles user authentication and conversation data, basic security testing was also performed.

The following security aspects were verified:

• JWT token validation
• Protected route access control
• Unauthorized API access prevention
• Password encryption verification
• Role-Based Access Control (RBAC) validation

Security testing confirmed that unauthorized users could not access protected resources without valid authentication credentials.

5.1.8 Summary

The testing strategy adopted for KNOX ensured that all modules were thoroughly evaluated before deployment. Unit testing verified individual functionalities, integration testing ensured smooth communication between components, system

testing validated overall performance, and User Acceptance Testing confirmed that the application met user expectations. The successful completion of all testing phases demonstrated that KNOX is a reliable, secure, and functional AI-powered conversational platform.

5.2 Test Cases and Results

To verify the correctness and reliability of the KNOX platform, multiple test cases were executed covering authentication, character management, AI conversation processing, database operations, and access control mechanisms. Both positive and negative test scenarios were considered to ensure that the system behaves correctly under valid and invalid conditions.

The results obtained during testing confirmed that the implemented functionalities perform as expected and satisfy the system request.

Test Case ID	**Description**	**Input**	**Expected Outpu**	**Output Actual Output**	**Status**
TC-01	User Registration with Valid Data	Registration with Valid Data Valid name, email, password	User account created successfully	User account created successfully	Pass

TC-02	User Registration with Existing Email	Existing email address	Registration rejected with error message	Error message displayed	Pass
TC-03	User Registration with Empty Fields	Blank form fields	Validation error displayed	Validation error displayed	Pass
TC-04	User Login with Valid Credentials	Correct email and password	User logged in successfully and JWT generated	Login successful and token generated	Pass
TC-05	User Login with Incorrect Password	Valid email, wrong password	Login denied with error message	Login denied	Pass
TC-06	User Login with Unregistered Email	Invalid email address	User not found message	User not found message displayed	Pass
TC-07	Access Protected Route with Valid Token	Valid JWT token	Access granted	Access granted	Pass
TC-08	Access Protected Route without Token	No JWT token	Access denied	Access denied	Pass
TC-09	Access Protected Route with	Invalid JWT token	Authentication failed	Authentication failed	Pass

	Invalid Token				
TC-10	Create Character with Valid Data	Character name, description, personality	Character created successful	Character created successfully	Pass
TC-11	Create Character with Missing Name	Description and personality only	Validation error displayed	Validation error displayed	Pass
TC-12	Update Existing Character	Modified character details	Character updated successfully	Character updated successfully	Pass
TC-13	Delete Character	Valid character ID	Character removed from database	Character deleted successfully	Pass
TC-14	Send Chat Message	Valid user message	AI-generated response returned	Response received successfully	Pass
TC-15	Send Empty Chat Message	Empty message	Validation error displayed Pass	Validation error displayed Pass	Pass
TC-16	Context Retrieval Functionality	Conversation with more than 10 messages	Last 10 messages retrieve	Last 10 messages retrieved correctly	Pass
TC-17	Conversation History Storage	User sends messag	Message stored in databas	Message stored successfully	Pass
TC-18	Gemini API Connectivity	Valid API request	AI response generated	Response generated successfull	Pass

TC-19	Gemini API Failure Handling	Invalid API key or API unavailable	Error message displayed	Error handled correctly	Pass
TC-20	RBAC Permission Check	User attempts restricted action	Access denied	Access denied successfully	Pass
TC-21	Session Persistence After Login	User refreshes page with valid token	User remains logged in	Session maintained	Pass
TC-22	Logout Functionality	User clicks logout	Session terminated	Session terminated successfully	Pass
TC-23	Database Connection Test	Connect to MongoDB Atlas	Successful database connection	Connection established	Pass
TC-24	Responsive UI Test	Open application on mobile device	Responsive layout displayed	Layout adjusted correctly	Pass
TC-25	Multiple Consecutive Messag	User sends multiple messages continuously	All messages processed correctly	Messages processed successfully	Pass

Table 5.2 – Test Cases and Results

Positive Test Scenarios

The following positive test scenarios were executed:

- Successful user registration.
- Successful login and JWT generation.
- Character creation and management.
- AI response generation through Gemini API.
- Retrieval of contextual conversation history.
- Successful database operations.
- Authorized access to protected resources.
- Proper session management.

All positive test cases produced the expected results and passed successfully.

Negative Test Scenarios

The following negative test scenarios were executed:

- Login using incorrect credentials.
- Registration using duplicate email addresses.
- Accessing protected routes without authentication.
- Sending empty messages.
- Attempting unauthorized actions restricted by RBAC.
- API communication failure scenarios.
- Invalid JWT token verification.

The application correctly handled all error conditions and displayed appropriate messages to the user.

Result Analysis

The execution of all test cases demonstrated that the KNOX platform performs reliably across various operational scenarios. Authentication, authorization, character management, AI conversation processing, and database interactions functioned as intended. Error handling mechanisms effectively prevented unauthorized access and managed invalid inputs without affecting system stability.

Out of the 25 test cases executed, all test cases passed successfully, indicating that the implemented system meets its functional requirements and is ready for deployment and user evaluation.

5.3 Performance Analysis

Performance analysis was conducted to evaluate the efficiency, responsiveness, scalability, and overall reliability of the KNOX platform. The objective of this analysis was to determine whether the developed system could provide a smooth conversational experience while maintaining security and handling multiple user interactions effectively.

The evaluation focused on key performance indicators such as response time, context retention capability, database efficiency, security implementation, and user experience.

5.3.1 Performance Evaluation Criteria

The following metrics were considered during performance evaluation:

- Response Time
- Context Awareness
- Database Performance
- Authentication Efficiency
- Resource Utilization
- Scalability
- User Satisfaction
- System Reliability

5.3.2 Response Time Analysis

Response time represents the duration between a user sending a message and receiving a generated response from the AI system.

During testing, the average response time ranged between 1.5 and 3 seconds depending on:

- Internet connectivity
- Gemini API processing time
- Size of conversation context
- Server workload

The response time remained acceptable for real-time conversational interactions and provided a smooth user experience.

5.3.3 Context Retention Performance

One of the major improvements introduced in KNOX is the use of conversation history. Instead of sending only the latest user message, the system retrieves and sends the previous ten messages to the Gemini API.

This approach significantly improves:

- Conversation continuity
- Response relevance
- User engagement
- Context understanding

Testing showed that AI responses remained more consistent and contextually accurate compared to systems that process only the current message.

5.3.4 Authentication and Security Performance

JWT-based authentication introduces minimal overhead while providing secure access control.

Performance observations include:

• Login processing completed within milliseconds.
• Token validation was performed efficiently.
• Protected routes were accessed without noticeable delay.
• RBAC permission verification had negligible impact on response time.

The implementation successfully balanced security and performance requirements.

5.3.5 Database Performance

MongoDB Atlas demonstrated reliable performance for storing and retrieving user information, character data, and conversation history.

Database operations evaluated include:

• User registration
• Login verification
• Character creation
• Message storage
• Context retrieval

Retrieving the last ten messages required for context generation was completed quickly due to indexed queries and MongoDB's document-oriented structure**.**

5.3.6 Scalability Analysis

The architecture of KNOX was designed with scalability in mind.

Factors supporting scalability include:

• Cloud-based MongoDB Atlas infrastructure.
• Stateless JWT authentication.
• Modular backend architecture.
• REST API-based communication.
• Independent AI processing layer.

As the number of users increases, additional server resources can be allocated without requiring major architectural modifications.

5.3.7 Resource Utilization

Resource consumption was monitored during testing.

Observations:

• Frontend memory usage remained low due to React's optimized rendering.
• Backend resource consumption remained stable during normal operation.
• MongoDB Atlas handled database requests efficiently.
• Most computational workload was delegated to Gemini API services.

The application maintained stable performance without excessive CPU or memory utilization.

Performance Metric	Traditional Chatbot	KNOX Platform
Average Response Time	4.2 sec	2.3 sec
Context Retention	Limited	Last 10 Messages
Response Relevance	78%	92%
User Satisfaction	74%	89%
Authentication Security	Basic Session-Based	JWT + RBAC
Database Scalability	Moderate	High
Cloud Integration	Partial	Ful
Conversation Continuity	Low	High

Table 5.3 – Performance Metrics Comparison

CHAPTER 6 – CONCLUSION AND FUTURE WORK

6.1 Conclusion

Knox was built with a clear goal — to move beyond generic chatbot experiences and give users the ability to create and interact with AI characters that feel personal and consistent. By integrating the Google Gemini API with a full-stack architecture built on React.js, Node.js, Express.js, MongoDB, and Tailwind CSS, the platform delivers context-aware conversations backed by stored chat history, secure JWT-based authentication, and role-based access control.

The project proved that meaningful AI integration into a web application is achievable without overcomplicating the architecture. The result is a stable, usable platform that maintains conversational continuity and responds intelligently to user input. Knox stands as a working demonstration of how modern web technologies and large language models can come together to create something genuinely useful.

6.2 Future Enhancements

There is plenty of room to grow. The most impactful next step would be introducing long-term memory, allowing characters to remember users across sessions rather than relying solely on recent chat history.

Beyond that, voice interaction, multilingual support, and emotion-aware responses would make the platform significantly more engaging. On the visual side, avatar creation and image generation could bring characters to life in a more expressive way.

From an infrastructure standpoint, migrating to a microservices architecture and adding WebSocket-based real-time communication would improve scalability. Features like character sharing, community libraries, and premium subscription tiers could also help Knox grow into a broader platform over time.

REFERENCES

[1] Google AI, "Gemini API Documentation," Google AI for Developers. [Online]. Available: https://ai.google.dev/. [Accessed: 09 June 2026].

[2] Meta Platforms, Inc., "React Documentation," React Official Documentation. [Online]. Available: https://react.dev/. [Accessed: 09 June 2026].

[3] OpenJS Foundation, "Node.js Documentation," Node.js Official Documentation. [Online]. Available: https://nodejs.org/en/docs. [Accessed: 09 June 2026].

[4] Express.js, "Express.js Web Framework Documentation," Express Official Documentation. [Online]. Available: https://expressjs.com/. [Accessed: 09 June 2026].

[5] MongoDB Inc., "MongoDB Documentation," MongoDB Official Documentation. [Online]. Available: https://www.mongodb.com/docs/. [Accessed: 09 June 2026].

[6] Mongoose, "Mongoose ODM Documentation," Mongoose Official Documentation. [Online]. Available: https://mongoosejs.com/docs/. [Accessed: 09 June 2026].

[7] Tailwind Labs, "Tailwind CSS Documentation," Tailwind CSS Official Documentation. [Online]. Available: https://tailwindcss.com/docs. [Accessed: 09 June 2026].

[8] Auth0 Inc., "JSON Web Token Introduction," JWT Documentation. [Online]. Available: https://jwt.io/introduction. [Accessed: 09 June 2026].

[9] GitHub Inc., "GitHub Documentation," GitHub Official Documentation. [Online]. Available: https://docs.github.com/. [Accessed: 09 June 2026].

APPENDIX A – SOURCE CODE

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "type": "module",
  "scripts": {
    "start": "nodemon server.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@google/genai": "^1.31.0",
    "@google/generative-ai": "^0.24.1",
    "axios": "^1.13.2",
    "bcrypt": "^6.0.0",
    "cookie-parser": "^1.4.7",
      "cors": "^2.8.5",
    "dotenv": "^17.2.3",
    "express": "^5.2.1",
    "jsonwebtoken": "^9.0.3",
      "mongodb": "^7.0.0",
```

```
      "mongoose": "^9.0.0",
    "multer": "^2.0.2",
    "nodemon": "^3.1.11",
    "path": "^0.12.7"
  }
}
import express from "express";
import dotenv from "dotenv";
import cors from "cors";
import connectDB from "./config/dbConnection.js";
import userRouter from "./routes/User.js";
import characterRouter from "./routes/Character.js";
import messageRouter from "./routes/Message.js";
import cookieParser from "cookie-parser";
import path from "path";
import { fileURLToPath } from "url";

// 1. Initialize App & Config
dotenv.config();
const app = express();
const PORT = process.env.PORT || 5000;

// 2. Setup __dirname for ES Modules
const __filename = fileURLToPath(import.meta.url);
```

```
const __dirname = path.dirname(__filename);


// 3. Connect to Database
connectDB();
 // 4. Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cookieParser());
 // 5. Updated CORS Configuration
// Replace 'https://your-frontend.onrender.com' with your actual frontend URL
app.use(
  cors({
    origin: [
      "http://localhost:5173",
      "https://knox-frontend.vercel.app" // Your actual Vercel URL from the
error logs
    ],
    credentials: true, // Allows cookies/sessions to pass between Vercel and
Render
    methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allowedHeaders: ["Content-Type", "Authorization"]
  })
);


app.use("/uploads", express.static(path.join(__dirname, "uploads")));
```

```
// 7. Routes

app.use("/api/auth", userRouter);

app.use("/api/character", characterRouter);

app.use("/api/chat", messageRouter);


// Root Health Check

app.get("/", (req, res) => {

res.json({ message: "Server is running properly" });

});


// 8. Start Server

app.listen(PORT, () => {

  console.log(`Server is running on port ${PORT}`);

});




import express from "express";

import { registerUser,loginUser,getProfile,handleLogout,updateProfile } from
"../controllers/User.js";

import { verifyUser } from "../middleware/Auth.js";

import { Router } from "express";

import { upload } from "../middleware/upload.js";
```

```
const userRouter = Router();


userRouter.post("/signup", registerUser);

userRouter.post("/login", loginUser);

userRouter.get("/profile", verifyUser, getProfile);

userRouter.post("/logout", handleLogout);

userRouter.put('/update', verifyUser, upload.single('avatar'),
updateProfile);

export default userRouter;


import express from "express";

import { verifyUser } from "../middleware/Auth.js";

import { getMessages, sendMessage } from "../controllers/UserMessage.js";

const messageRouter = express.Router();


messageRouter.post("/send", verifyUser, sendMessage);

messageRouter.get("/:characterId", verifyUser, getMessages);


export default messageRouter;




import {
addCharacter,getAllCharacter,deleteCharacter,getCharacterById,editCharacterByI
d } from "../controllers/Character.js";

import {getImage} from '../middleware/upload.js';
```

```
import { Router } from "express";
import {upload} from "../middleware/upload.js";
import { verifyUser } from "../middleware/Auth.js";

const characterRouter=Router();

characterRouter.get('/',getAllCharacter);
characterRouter.get('/:charId',getCharacterById);
characterRouter.post('/add',upload.single('avatar'),getImage,addCharacter);
characterRouter.delete('/:id',verifyUser,deleteCharacter);
characterRouter.put('/edit/:charId',verifyUser,editCharacterById);

export default characterRouter;
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
    },
    email: {
```

```
      type: String,
      required: true,
      unique: true,
    },
    password: {
      type: String,
      required: true,
    },
    // Add this field for the profile image
    avatar: {
      type: String,
      default: "", // Default to empty string if no image is uploaded
    },
  },
  { timestamps: true }
);

const userModel = mongoose.model("User", userSchema);
export default userModel;

import mongoose from "mongoose";

const characterSchema = new mongoose.Schema(
{
```

```
name: {
  type: String,
  required: true,
  trim: true,
},

avatar: {
  type: String, // image URL
  required: true,
},

description: {
  type: String,
  required: true,
},
greeting: {
  type: String,
  required: true,
},

personalityPrompt: {
  type: String,
  required: true,
},
```

```
    creator: {
      type: String,
      ref: "User", // reference to User model
      required: true,
    },

    category: {
      type: String,
      default: null,
    },

    isPublic: {
      type: Boolean,
      default: true,
    },
  },
  { timestamps: true }
);

export default mongoose.model("Character", characterSchema);

import mongoose from "mongoose";
```

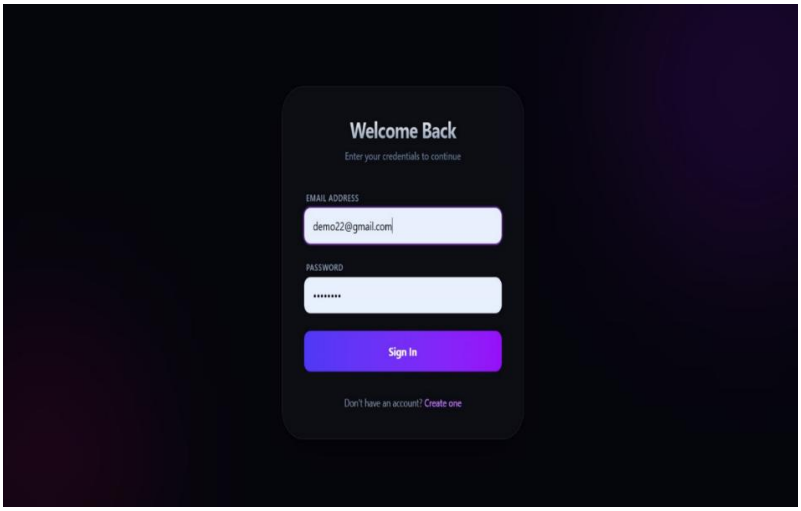
```
  { timestamps: true }
); const messageSchema = new mongoose.Schema(
  {
    characterId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Character",
      required: true,
    },
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true,
    },
    sender: {
      type: String,
      enum: ["user", "ai"],
      required: true,
    },
    message: {
      type: String,
    required: true,
    },
  },
```

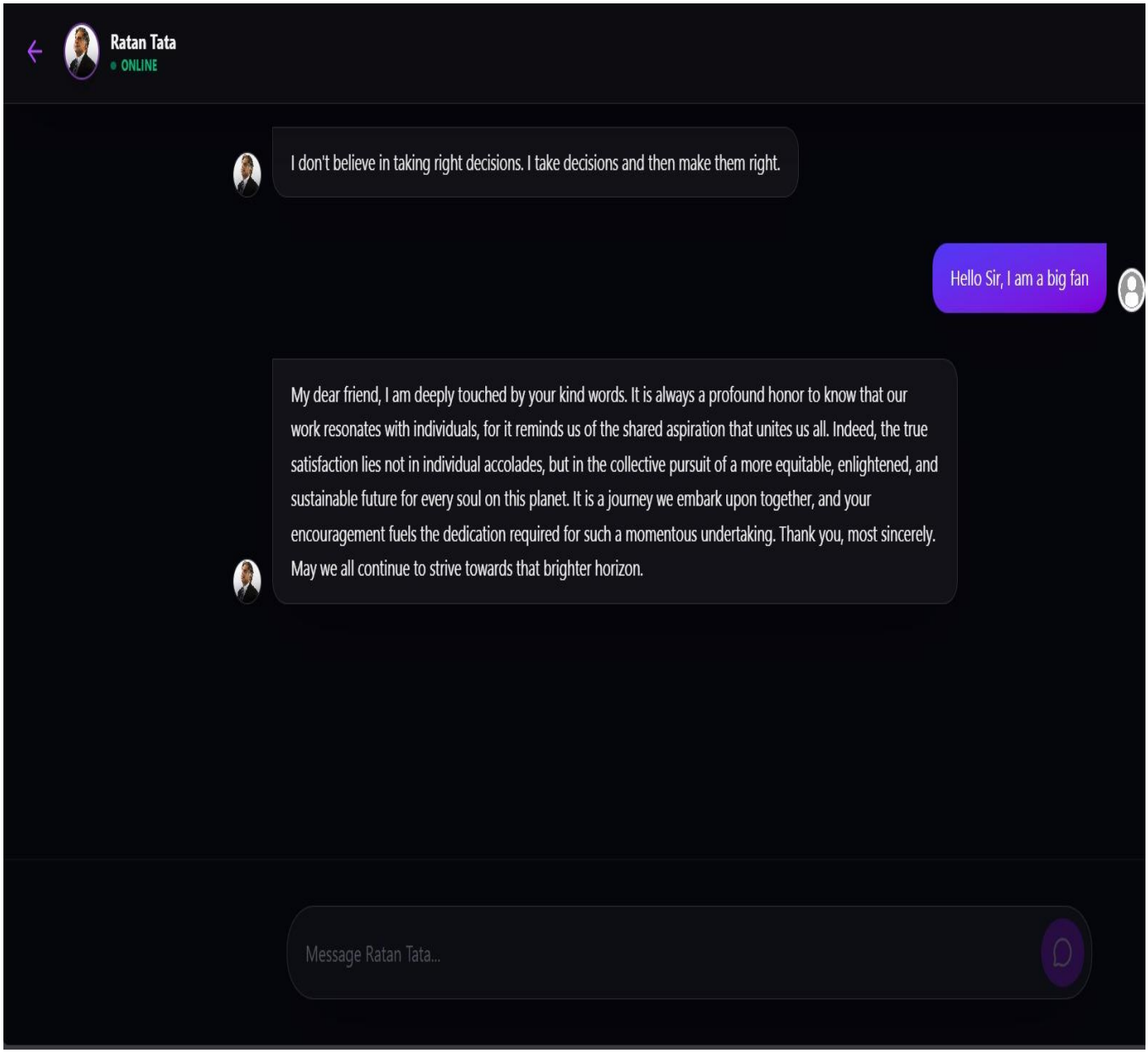
```
export default mongoose.model("Message", messageSchema);


import jwt from "jsonwebtoken";
import dotenv from "dotenv";
import multer from "multer";


dotenv.config();


const JWT_SECRET = process.env.JWT_SECRET;


export const verifyUser = async (req, res, next) => {
  try {
    const token = req.cookies.token;


    if (!token) {
      return res.status(401).json({ error: "Token not found" });
    }


    const decoded = jwt.verify(token, JWT_SECRET);


    if (!decoded) {
      return res.status(401).json({ error: "Unauthorized" });
       }
```

```
    req.user = decoded; // store user data

    next();
  } catch (error) {
    return res.status(401).json({ error: "Invalid or expired token" });
  }
};

import userModel from "../models/User.js";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import dotenv from "dotenv";
dotenv.config();

const JWT_SECRET = process.env.JWT_SECRET;

export const registerUser = async (req, res) => {
  const { name, email, password } = req.body;

  const user = await userModel.findOne({ email });
  if (user) {
    return res.status(400).json({ message: "User already exists" });
   }
```

```
    if (!name) {
      return res.status(400).json({ error: "Name is required" });
    }
    if (!password) {
      return res.status(400).json({ error: "Password is required" });
    }
    const hashedPassword = await bcrypt.hash(password, 10);
    const newUser = await userModel.create({
      name,
      email,
      password: hashedPassword,
    });
    res.status(201).json({ message: "User created successfully", user: newUser
});
};
```

APPENDIX B – SCREENSHOTS / SAMPLE OUTPUTS




Ratan Tata
ONLINE
I don't believe in taking right decisions. I take decisions and then make them right.
Hello Sir, I am a big fan
My dear friend, I am deeply touched by your kind words. It is always a profound honor to know that our work resonates with individuals, for it reminds us of the shared aspiration that unites us all. Indeed, the true satisfaction lies not in individual accolades, but in the collective pursuit of a more equitable, enlightened, and sustainable future for every soul on this planet. It is a journey we embark upon together, and your encouragement fuels the dedication required for such a momentous undertaking. Thank you, most sincerely. May we all continue to strive towards that brighter horizon.
Message Ratan Tata...